CO7207 Generative Development 2026: Homework (HW1)

UML Modelling and Programming with EMF (Eclipse Modelling Framework)

Due date: Friday 27th February 2026 @ 12:00 mid-day

Submission: Blackboard: Assessment and Feedback >> Homework (HW1)

Weight: 50%

Plagiarism, Late Submission and Marking

You will be working in small teams on this assignment. To avoid plagiarism or collusion, you **should not be working with anybody other than your group** on this coursework. Any plagiarism or collusion will result in a serious penalty.
https://le.ac.uk/policies/regulations/senate-regulations/senate-regulation-11/academic

Late submission will result in a penalty as per the senate regulations.
https://le.ac.uk/policies/regulations/senate-regulations/senate-regulation-7

Introduction

In this homework you will develop software models for a system designed to support a business. You will choose the business and the main functionality of the system as part of Task 1 below. This choice is important as the models you will develop in the remaining tasks must match with your choice.

Setup and Important Guidance

A video that explains this coursework in detail is linked on Blackboard. Download the "gendev.hw1.zip" file available on Blackboard. Import this project into Eclipse (via "File -> Import… -> Existing Projects into Workspace -> (o) Select archive file -> … -> Finish"). Solve all tasks within this project. After completing all tasks, you can then export this project as a zip file, and upload to Blackboard.

- Do not delete any files from "gendev.hw1.zip" – if you do, the best way to fix this is to re-download a fresh copy of the zip and begin again
- When you have completed all tasks in the "gendev.hw1" project – you should then export this as a zip file (via "File -> Export -> General -> Archive File -> To archive file -> [enter a name] -> Finish")
- The name of your zip must be "A-B-HW1.zip" – where "A" = the student email ID of member 1, and "B" = the student email ID of member 2. For example, if jbh13 and jbh14 are working together, their submitted zip file should be named "jbh13-jbh14-HW1.zip". If working individually, only A is required. Add a third if working as a group of 3.
- A single zip file must be submitted (you can submit as many times as you like, but only the last attempt will be marked)
- Files that do not follow the naming requirement, or that do not compile, will not be marked
- The "EarlyFeedback" file (in the "src" folder) can be used to help you check that the submission is correct – but pay attention to what it outputs!

You need to describe the contribution of each member of your group, and your use of AI tools, in the file "Contribution-and-AI.txt" in the folder "contribution-and-ai" of the "gendev.hw1.zip". In this file, you need to explain which member worked on which component of the submission and state the percentage of contribution.

Task 1: Critically using Copilot for Description [20 marks]

Choose a business that you will model. To do this, use Copilot (via your university account) to discuss options, and to explore your chosen business more. During your discussion, you should explore key stakeholders, a list of requirements for your system and plans for a class diagram.

Being critical on AI output and using skills in prompt engineering, use Copilot to generate a 100-200 word description of your chosen business. This should list the stakeholders (e.g. customers, staff, etc), the business entities (e.g., your classes), and both the use cases and actions you model later (please read ahead to be aware of requirements).

After you have done this, reflect on your Copilot discussion. Write a 200-300 word critical evaluation of your use of Copilot. In this discussion, explain things that Copilot did well, not so well, how you used critical evaluation of its output to improve prompts, things that Copilot may have missed, etc. Link directly in one entry on the "Generative AI issues" padlet (linked on Blackboard) – using a direct quote in quotation marks.

Submission

- Place your AI generated description into the file "task1-Description/description.txt"
- Place your critical evaluation of your use of Copilot into the file "task1-Description/critical.txt". This file should also contain a Copilot Pages link that includes your complete Copilot chat (including all prompts and responses).
- All word counts are subjective to the usual +/- 10% rule – with a penalty applied after that point.

Marking Criteria

- An appropriate project has been chosen, and the Copilot generated description is reasonable and submitted correctly **[2 marks]**
- Key stakeholders are identified (and consistent with later) **[2 marks]**
- Key use cases and actions are identified (and consistent with later) **[2 marks]**
- Critical evaluation is appropriate and reasonable **[2 marks]**
- Clear written explanation of how you used prompts to improve / strengthen AI output with examples **[4 marks]**
- Clear and appropriate discussion of the use of Copliot as part of software engineering. Personal and specific reflection on what went well, not so well and how you might use this in the future **[5 marks]**
- Link to the padlet is made clearly and specifically, with a direct quote and a critical comment of it given **[3 marks]**

Task 2: Generate/Improve Behaviour Models [30 marks]

Part a) Use Case Diagram

Use Copilot to generate a use case model for your chosen business. This should be in the format required for PlantUML (https://plantuml.com/) and consist of at least 4 actors and 10 use cases. This generated model should be submitted as "Copilot-Use-Case".

Using this as the basis, alter and improve this use case model such that:

- All use case names are meaningful and active
- Correct and non-trivial participation of actors in Use Cases (at least one example of more than one actor involved in a Use Case)
- Correct and meaningful use of at least one <<extend>> and at least one <<include>>
- Correct and meaningful use of inheritance for Use Cases and for actors

Produce a 200-300 word description and critical evaluation of the use of Copilot to generate this diagram. What did it do well, and not so well? What did you improve, and why?

Part b) Activity Diagram

Use Copilot to generate an activity diagram for your chosen business. This should be in the format required for PlantUML (https://plantuml.com/) and consist of at least 10 meaningful actions. This generated model should be submitted as "Copilot-Activity".

Using this as the basis, alter and improve this activity diagram such that:

- All action names are meaningful and active
- At least 2 correct uses of decision and merge nodes including guards on outgoing arrows
- At least one correct use of a fork and a join node
- One use of an accept event, and one use of a send event

Produce a 200-300 word description and critical evaluation of the use of Copilot to generate this diagram. What did it do well, and not so well? What did you improve, and why?

Submission

- Add the Copilot generated PlantUML code for use case and activity diagrams to the folders "task2_Usecase-Diagram" and "task2-Activity-Diagram" respectively. Place the code into "Copilot-UseCase-PlantUML.txt" and "Copilot-Activity-PlantUML.txt" respectively.

CO7207 Generative Development 2026: Homework (HW1)

- Add your improved use case and activity diagrams to the folders “task2_Usecase-Diagram” and “task2-Activity-Diagram” respectively. Place the code into “Critical-Final-UseCase-PlantUML.txt” and “Critical-Final-Activity-PlantUML.txt” respectively.
- Add a PNG image screenshot of both versions of your Use case, and the activity, diagrams. Label these as “Copilot-Usecase.png”, “Critical-Final-Usecase.png”, “Copilot-Activity.png” and “Critical-Final-Activity.png”. Place these into the folders “task2-Activity-Diagrtam” and “task2-Usecase-Diagram” respectively.
- Submit a Copilot Pages link that includes your complete Copilot chat (including all prompts and responses). Place (and explain) this into the “contribution-and-ai.txt” file.
- Place your 200-300 word critical evaluation of your Use Case and 200-300 word critical evaluation of your Activity Diagram into the files “how-we-were-critical.txt” in “task2-Usecase-Diagram” and “task2-Activity-Diagram” respectively.

Marking Criteria

- A correct and appropriate use case model (all use case names are meaningful and active (2 mark), 2 correct uses of decision and merge (2 marks), 1 correct use of fork and join (1 mark), 1 correct use of accept and send (2 mark) **[7 marks]**
- A correct and appropriate activity diagram (all action names are meaningful and active (2 mark), non-trivial participation of actors (1 mark), 1 correct use of extends and includes (2 marks), 2 correct uses of inheritance (one between actors and one between use cases) (2 mark) **[7 marks]**
- Critical evaluation of how you improved each model initially produced by Copilot. Clear and specific examples (at least 3) of things you improved and why (6 marks), and clear reflection and specific examples of things Copilot did well (2 marks) **[8 marks for Use case, 8 marks for Activity – including marks for consistency]**

Task 3: Structural Model [30 marks]

The following tasks will require the use of Eclipse and the Eclipse Modelling Framework. The previous lab sessions will help you with learning the necessary techniques.

Part a) Class Diagram

Using EMF, implement the class diagram that you have co-created with Copilot. Use role names and multiplicities on all association ends. **[10 marks]**

Part b) OCL Invariants

Write a definition of three meaningful and appropriate OCL class invariants. The names of invariants must start with "inv1", "inv2", and "inv3" respectively. In words, you should explain the intended purpose of each invariant and discuss both the satisfies and violates instance you will use in Task 4. Read ahead in the marking criteria for required concepts and structures. **[10 marks]**

Now add those three meaningful OCL class invariants to the Ecore model. Do this using the OCLinEcore editor (see here for details). The names of invariants must start with "inv1", "inv2", and "inv3" respectively (matching your definition). **[10 marks]**

Submission

- Add your invariants description into the file "task3-Invariants/invariant-description.txt"
- Create your class diagram in the existing file "model/hw1.aird"
- Add the OCL invariants to "model/hw1.ecore" via the OCLinEcore editor (names beginning "inv1", "inv2", and "inv3" respectively).

Marking Criteria

- Clear and appropriate explanation of each invariant (clear explanation of the intention, clear explanation of both a satisfies and violates instance – consistent with those used later) (3 marks each). Followed naming convention (1 mark) **[10 marks]**
- Class Diagram
  - Identify at least 6 classes with correct and meaningful class names – appropriate to your chosen business (1 mark each) **[2 marks]**
  - Use inheritance appropriately **[1 mark]**
  - Use at least one abstract class appropriately **[1 mark]**
  - Use of meaningful/appropriate attributes – at least one per class **[2 marks]**
  - At least 4 correct associations with association role names and non-trivial multiplicities (not all * or 0..*) **[4 marks]**
- OCL class invariants

- Use attributes **[1 mark]**
- Use of navigation leading to a single instance **[1 mark]**
- Use of navigation leading to a set of instances **[1 mark]**
- Use navigation via multiple associations **[1 mark]**
- Use of if-then-else **[1 mark]**
- Use of comparison operator (e.g. “<” or “>”) **[1 mark]**
- Use of size(), exists(), forall() (1 mark each) **[3 marks]**
- Meaningful invariants used **[1 mark]**

Task 4: Instances [20 marks]

You will now continue your use of EMF, this time focusing on instances.

Part a) Instance of Ecore Model (Object Diagram)

Using EMF API (and not the editor plugin), add java code to "src/gendev.hw1/Task4.java" that creates an instance of the Ecore Model (in other words, an Object Diagram). This should have at least 6 objects and 6 links and should be saved into XMI format. **[8 marks]**

Part b) Instances of OCL Invariants

Using the Dynamic Instance Editor, create two instances per OCL invariant. One instance must satisfy the OCL invariant, while the other must violate the OCL invariant. All instances must be valid in the Class Diagram when ignoring the OCL invariants, and must be consistent with the invariants described in Task 3. **[12 marks]**

Submission

- Instances of Ecore Model (Object Diagram)
  - Implement the code in "src/gendev/hw1/Task4.java"
  - Save the XMI to "task4_Instances/instance.xmi"
- Instances of OCL Invariants
  - "task4_Instances/inv1_sat.xmi" (satisfying invariant "inv1…")
  - "task4_Instances/inv1_fail.xmi" (violating invariant "inv1…")
  - "task4_Instances/inv2_sat.xmi" (satisfying invariant "inv2…")
  - "task4_Instances/inv2_fail.xmi" (violating invariant "inv2…")
  - "task4_Instances/inv3_sat.xmi" (satisfying invariant "inv3…")
  - "task4_Instances/inv3_fail.xmi" (violating invariant "inv3…")

Marking Criteria

- Code to create the instance entered into "src/gendev.hw1/Task4.java" **[6 marks]**
- Code to save the instance into XMI **[2 marks]**
- Total of 6 correct OCL instances (2 marks each) **[12 marks]**